

Sistemas Operativos

-Trabalho Prático 2025/2026-

Programação em C para UNIX



Artur Meireles de Sousa Falcão

-2024130485

Gabriel Cardoso Rodrigues

-2024133397-

Índice

Introdução -----	3
Arquitetura Global do Sistema -----	3
Mecanismos de Comunicação -----	3
Estratégia da Implementação -----	4
Estrutura Interna dos Programas -----	4
Justificação das opções tomadas -----	5
Tabela de Funcionalidades Requeridas -----	6
Conclusão -----	6

Introdução

O presente relatório descreve o desenvolvimento de uma plataforma distribuída, implementada em C para UNIX, destinada à simulação de um sistema de transporte baseado numa frota de veículos autónomos.

A plataforma é constituída por três aplicações independentes que comunicam entre si: controlador, cliente e veículo.

O objetivo do projeto é colocar na prática os conhecimentos lecionados na unidade curricular de Sistemas Operativos, incluindo criação e gestão de processos, concorrência, comunicação entre processos e manipulação adequada de recursos do sistema.

Arquitetura Global do Sistema

A plataforma é composta por três processos distintos:

- **Controlador:**
Responsável pela gestão global da plataforma, agendamento de serviços, lançamento de veículos, controlo da frota e comunicação com os clientes. Serve simultaneamente como servidor (recebe pedidos dos clientes) e como interface para o administrador.
- **Cliente:**
Cada utilizador executa um cliente que permite registar um username, agendar e consulta serviços, cancelar pedidos e receber mensagens tanto do controlador como do veículo. Este utiliza um named pipe dedicado para comunicação direta com o veículo e outro (o global) para o controlador.
- **Veículo:**
Processo filho criado pelo controlador para realizar um serviço. Este envia telemetria para o controlador via stdout redirecionado para um pipe anónimo, e envia mensagens diretamente ao cliente por meio do named pipe privado de cada utilizador.

Mecanismos de Comunicação

Ligação	Mecanismo	Sentido
Cliente – Controlador	Named pipe global (empresa_taxis)	Pedidos
Controlador – Cliente	Named pipe privado do cliente (cli_<user>)	Respostas/notificações
Controlador – Veículo	Argumentos de linha de comandos	Configuração inicial
Veículo – Controlador	Pipe anónimo (stdout redirecionado)	Telemetria
Veículo – Cliente	Named pipe privado do cliente	Notificações

Estratégia da Implementação

O **Controlador** funciona como o processo principal e coordena toda a lógica:

- Mantém tabela de serviços agendados;
- Mantém lista de utilizadores ativos;
- Monitoriza e atualiza estados de veículos;
- Lança processos veículo no momento correto;
- Implementa um relógio simulado que avança a cada segundo;
- Recebe comandos do administrador através do stdin;
- Recebe pedidos dos clientes por um named pipe global.

O **Cliente** como interface do utilizador:

- Cria o seu próprio FIFO privado;
- Regista-se no controlador;
- Usa select() para permitir entrada simultânea de:
 - Comandos escritos pelo utilizador,
 - Mensagens recebidas do controlador ou veículo.
- Encaminha pedidos para o computador (agendar, cancelar, consultar);
- Recebe notificações diretas do veículo.

Cada **Veículo** é um processo lançado com fork() e execlp():

- Envia telemetria para o controlador (via stdout => pipe anónimo);
- Envia ao cliente:
 - Aviso de início de serviço,
 - Percentagens de progressão,
 - Conclusão ou cancelamento.
- Termina ao concluir o serviço ou ao receber SIGUSR1.

Estrutura Interna dos Programas

Controlador:

- Gestão de Utilizadores
 - Vetor estático com máximo de 30 utilizadores;
 - Verificação de unicidade de usernames;
 - Adição e remoção controlada;
 - Notificação de terminação a todos os clientes.
- Gestão de Serviços
 - ID, utilizador, hora agendada;
 - Local de partida;
 - Distância total;
 - Estado atual (Agendado, Espera, Em trânsito, Cancelado, Concluído);
 - PID do veículo associado;
 - Progresso percentual.

- **Gestão de Frota**
 - Número máximo de veículos simultâneos definidos por NVEICULOS;
 - Controlo do número de veículos em execução simultânea;
 - Lançamento de veículos através de fork() e execlp();
 - Captura de telemetria via leitura não bloqueante dos pipe anónimos.
- **Relógio Simulado**
 - Incrementa automaticamente a cada segundo real;
 - Gera eventos de lançamento de veículos na hora certa.
- **Processamento assíncrono**
 - Leitura não bloqueante do stdin do administrador;
 - Leitura não bloqueante do FIFO global dos clientes;
 - Leitura não bloqueante dos pipes dos veículos.

Cliente:

- **Responsabilidades:**
 - Registrar-se enviando REGISTER <username>;
 - Aguardar aceitação pelo controlador;
 - Usa select() para ler simultaneamente consola e FIFO;
 - Encaminhar comandos para o controlador;
 - Receber mensagens do controlador e do veículo;
 - Sair ordenadamente (limpando FIFO).
- **Comandos:**
 - agendar
 - consultar
 - cancelar
 - terminar

Veículo:

- Recebe argumentos: FIFO do cliente, distância, local, username e ID do serviço;
- Envia mensagem inicial ao cliente quando “chega”;
- Imprime no stdout (capturado pelo controlador):
 - Início do serviço,
 - Percentagem a cada 10%,
 - Conclusão ou cancelamento.
- Termina ao receber SIGUSR1.

Justificação das Opções Tomadas

- I/O não bloqueante no controlador:
Justificado pelo enunciado que proíbe select() no controlador, e pela necessidade de responder a múltiplas fontes de input em simultâneo.
- Uso de pipes anónimos para telemetria
É o mecanismo mais leve e adequado para redirecionar stdout de um processo filho.
- Um FIFO por cliente
Evita conflitos, simplifica endereçamento e permite comunicação direto veículo-cliente.

- Estados explícitos nos serviços
Facilita monitorização, listagens e controlo da frota.
- Simulação de tempo independente do sistema
Permite testar funcionalidades sem depender da hora real.

Tabela de Funcionalidades Requeridas

Funcionalidade	Estado	Justificação
Registo de utilizadores únicos	Cumprido	FIFO global + validação de duplicados
Limite de 30 utilizadores	Cumprido	Vetor estático e verificação
Agendamento de serviços	Cumprido	Guardado em estrutura própria
Consulta de serviços	Cumprido	Cliente solicita e controlador responde
Comunicação cliente-servidor via FIFO	Cumprido	Cancela agendados e em execução (via SIGUSR1)
Comunicação controlador-cliente via FIFO privado	Cumprido	Named pipe global
Comunicação veículo-controlador via stdout	Cumprido	FIFO por utilizador
Lançamento de veículos no tempo correto	Cumprido	Pipe anónimo
Limite de veículos no tempo correto	Cumprido	Relógio simulado
Limite de veículos simultâneos (NVEICULOS)	Cumprido	Contagem antes do lançamento
Mensagens ao cliente pelo veículo	Cumprido	Via FIFO privado
Percentagens a cada 10%	Cumprido	Loop progressivo com passo de 10%
Cancelamento do veículo via sinal	Cumprido	SIGUSR1 tratado no veículo
Comandos do administrador	Cumprido	listar, utiliz, frota, cancelar, km, hora, terminar
Terminação ordeira e limpeza	Cumprido	Unlink dos FIFOs, encerramento de processos

Conclusão

A implementação desenvolvida cumpre todos os requisitos funcionais e técnicos descritos no enunciado, aplicando corretamente os mecanismos estudados na unidade curricular de Sistemas Operativos.

O trabalho demonstra domínio prático de ferramentas do UNIX, tais como named pipes, pipes anónimos, processos, sinais e controlo explícito de I/O.

A plataforma funciona conforme especificado, sendo capaz de gerir múltiplos utilizadores, serviços e veículos em paralelo, cumprindo os limites e restrições impostas.